

DISEÑO DE SOFTWARE · DÉCIMO AÑO

Diseño Arquitectónico de Software

Cómo se planean por dentro las páginas y aplicaciones web

Géneros

Estilos

Patrones

Evaluación

Mapeo de datos

El sistema que vamos a diseñar

Será nuestro ejemplo base; en algunas partes lo compararemos con otros sistemas

El requerimiento

El colegio necesita un sistema web donde cada docente registre las notas de sus grupos y el sistema calcule solo la nota final, tomando en cuenta el porcentaje que vale cada rubro (tareas, pruebas, trabajo cotidiano).

¿Qué debe hacer?

- 1 Registrar la nota de cada estudiante en cada rubro.
- 2 Aplicar el porcentaje que vale cada rubro.
- 3 Calcular y mostrar el promedio final.
- 4 Guardar todo para poder consultarlo después.

Cómo se conecta por dentro

Docente

digita las notas en la pantalla



Interfaz

los formularios y las boletas



Lógica

aplica los porcentajes y calcula



Base de datos

guarda notas y promedios

Todo esto se ejecuta en el servidor del colegio, y el docente entra desde el navegador.

¿Qué es la arquitectura y por qué importa?

El diseño interno de un programa, decidido antes de escribir el código

La arquitectura de software es la forma en que se organizan las partes de un programa y la manera en que esas partes se comunican entre ellas.

1

Componentes

Las partes que hacen el trabajo.

En el sistema de notas: la interfaz, la lógica de cálculo, la base de datos y el servidor

2

Relaciones

Cómo se conectan las partes y se pasan la información.

En el sistema de notas: la interfaz le pide el promedio a la lógica, y la lógica consulta la base de datos

3

Reglas

Lo que sí y lo que no se puede hacer en ese diseño.

En el sistema de notas: la interfaz nunca consulta la base de datos por su cuenta

¿Por qué es importante?

1

Se decide al inicio

Es la primera decisión grande del proyecto: todo lo demás se construye encima de ella.

2

Arreglar después cuesta

Cambiarla a mitad del proyecto obliga a reescribir código que ya funcionaba.

3

El equipo trabaja coordinado

Todos saben en qué parte va cada pedazo de código y nadie hace doble trabajo.

4

Define la calidad

De ella dependen la velocidad, la seguridad y qué tan fácil es agregar cosas nuevas.

Un error de arquitectura puede obligar a rehacer el proyecto entero.

Descripciones arquitectónicas

El documento donde queda escrita la arquitectura del sistema

La descripción arquitectónica es el documento que explica la arquitectura. Se organiza en vistas: cada vista muestra el sistema desde un ángulo distinto y está pensada para una persona distinta del proyecto.

1

Vista lógica

Qué partes tiene el sistema y qué hace cada una.

Le sirve a: Programadores

Sistema de notas: la interfaz, la lógica y la base de datos

2

Vista de procesos

Qué ocurre mientras el programa está funcionando.

Le sirve a: Equipo técnico

Sistema de notas: qué pasa al pulsar el botón Calcular

3

Vista física

En qué computadoras o servidores vive cada parte.

Le sirve a: Encargado de redes

Sistema de notas: el servidor del colegio

4

Vista de usuario

Los pasos que hace la persona que usa el sistema.

Le sirve a: Cliente y usuarios

Sistema de notas: el docente entra, digita y revisa el promedio

Además siempre se escribe: el objetivo del sistema, las decisiones importantes que se tomaron, el motivo de cada decisión y las reglas que nadie debe romper.

Géneros y estilos arquitectónicos

Dos preguntas distintas sobre un mismo sistema

GÉNERO

¿A qué campo pertenece el sistema?

Es la categoría o el campo al que pertenece un sistema. Importa porque cada campo exige cosas distintas: dos sistemas del mismo género se parecen por dentro, y de géneros distintos casi nunca.

Educación • sistema de notas: no puede perder una nota

Comercio electrónico • tienda en línea: pagos seguros y muchas visitas

Salud • expediente médico: nunca pierde un dato

Transporte • app de buses: horarios y rutas al instante

Financiero • banca en línea: seguridad por encima de todo

Videojuegos • juego en red: velocidad y buenos gráficos

ESTILO

¿De qué forma están organizadas sus partes?

Es una forma conocida de organizar un sistema completo. Todo estilo define cuatro cosas:

1. **Componentes:** las piezas que hacen el trabajo
2. **Conectores:** por dónde viaja la información
3. **Restricciones:** las reglas de cómo se pueden unir
4. **Modelo semántico:** cómo se entiende el sistema ya armado

Estilos clásicos

Centrado en datos: todo gira alrededor de una base de datos central

Flujo de datos: la información pasa de una pieza a la siguiente

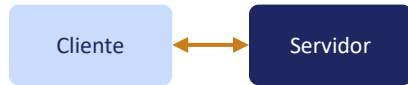
Llamada y retorno: un programa principal llama a otras partes y espera

Por capas: niveles apilados; cada nivel solo llama al de abajo

Orientado a objetos: cada pieza guarda sus datos y acciones, y se comunican

Los estilos más usados en desarrollo web

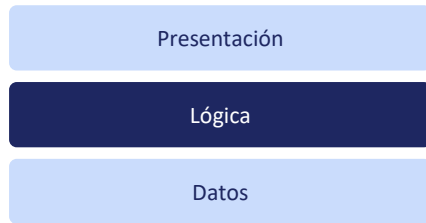
Qué es cada uno y en qué tipo de sistema conviene más



Cliente – Servidor

El navegador (cliente) pide y el servidor responde. Es la base de toda la web.

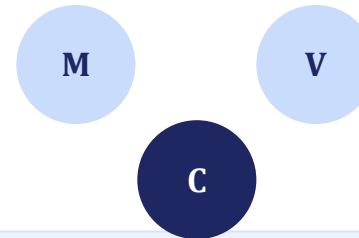
¿Dónde va mejor? en cualquier sitio web, como el sitio del colegio: siempre hay quien pide y quien responde.



Por capas

Presentación, lógica y datos en tres niveles; cada nivel solo habla con el de al lado.

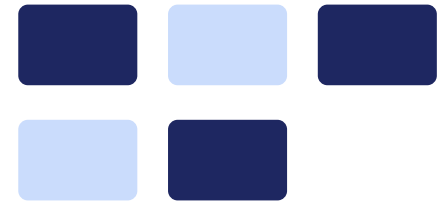
¿Dónde va mejor? en sistemas de una sola institución: el de notas, el de biblioteca, el de matrícula.



MVC

Modelo (los datos), Vista (lo que se ve) y Controlador (el que coordina) en piezas aparte.

¿Dónde va mejor? en sistemas con muchas pantallas distintas, como una tienda en línea.



Microservicios

El sistema se parte en servicios pequeños e independientes que se comunican.

¿Dónde va mejor? en plataformas enormes, como una red social o un servicio de streaming.

Patrones arquitectónicos

Soluciones ya probadas para problemas que se repiten siempre

Hay problemas que aparecen en casi todos los sistemas. Cuando alguien encuentra una buena solución, la escribe y los demás la reutilizan: eso es un patrón. Cada patrón nace de un problema concreto. El **estilo** organiza el sistema completo; el **patrón** resuelve una parte específica dentro de él.

Repositorio

Un solo lugar guarda toda la información y las demás partes le piden los datos a él.

Problema: cada parte guardaba lo suyo y los datos se contradecían.

¿Dónde se nota más? un sistema de biblioteca: un solo catálogo que consultan préstamos, multas y búsquedas.

Publicar y suscribirse

Una parte avisa que algo pasó y todas las interesadas se enteran solas.

Problema: avisarle uno por uno a cada componente.

¿Dónde se nota más? una app de mensajería: alguien escribe y a todo el grupo le llega el aviso solo.

Intermediario

Una pieza en el medio recibe todas las peticiones y las reparte.

Problema: demasiadas conexiones directas y desordenadas.

¿Dónde se nota más? una app de buses: una sola pieza consulta a varias empresas y devuelve los horarios.

Persistencia

Se separa el código que guarda en la base de datos del resto del programa.

Problema: cambiar de base de datos obligaba a rehacer todo.

¿Dónde se nota más? el sistema de notas: el código que guarda está aparte, por si cambian de base de datos.

Género, estilo y patrón: ¿en qué se diferencian?

Las tres palabras se parecen, pero responden preguntas distintas

GÉNERO

¿Para qué campo es?

Es la categoría del sistema según el campo al que sirve.

Ejemplos:

- Comercio electrónico
- Sistema financiero
- Videojuegos
- Salud

ESTILO

¿Cómo se organiza?

Es la forma general de acomodar las piezas del sistema completo.

Ejemplos:

- Cliente – servidor
- Por capas
- Flujo de datos
- Orientado a objetos

PATRÓN

¿Cómo resuelvo esto?

Es una solución ya probada para un problema puntual dentro del sistema.

Ejemplos:

- Repositorio
- Publicar y suscribirse
- Intermediario
- Persistencia

En el sistema de notas: su género es educación · su estilo es por capas · y usa el patrón de persistencia para guardar las notas.

El diseño arquitectónico paso a paso

Y qué son los arquetipos, las piezas base de todo el diseño

Los cuatro pasos del diseño

1

Ver el sistema desde afuera

Se define qué hace el sistema y con qué otros sistemas o personas se relaciona.

2

Ver el sistema desde adentro

Se identifican las piezas internas y qué le toca hacer a cada una.

3

Describir las piezas

Se detalla cada componente y cómo se conecta con los demás.

4

Ubicar cada parte en su servidor

Se decide en qué servidores o computadoras va a vivir cada parte.

Los arquetipos

Un arquetipo es un tipo de pieza que se repite en muchos sistemas distintos. Son pocos, y con ellos se describe casi cualquier arquitectura.

Nodo

una pieza que agrupa varias funciones relacionadas

Detector

recibe datos que entran al sistema

Indicador

muestra al usuario el estado o el resultado

Controlador

coordina a los demás y ordena qué se ejecuta

En un sistema de biblioteca: el escáner del código es el detector, la pantalla de disponibilidad es el indicador y el módulo de préstamos es el controlador.

Evaluar diseños alternativos

Casi nunca hay una sola arquitectura posible: hay que comparar y elegir

Para un mismo sistema se pueden proponer dos o tres arquitecturas distintas. Evaluar es compararlas con criterios claros y escoger con argumentos, no por gusto.

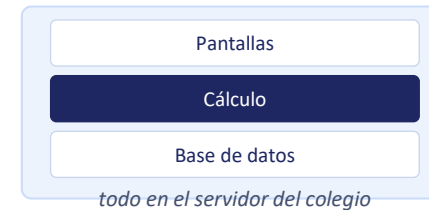
Los pasos

- 1 Reunir los requisitos y las cualidades que el sistema debe cumplir.
- 2 Proponer dos o más arquitecturas posibles.
- 3 Calificar cada una según los criterios elegidos.
- 4 Escoger la mejor y anotar por qué se descartaron las otras.

Ojo: la mejor arquitectura no es la más bonita, es la que cumple mejor lo que el sistema necesita.

Ejemplo: dos formas de armar el sistema de notas

A Un solo programa, en tres capas



- Fácil de construir
- Barato
- Si crece mucho, se pone lento

B Tres programas separados



cada uno en su propio servidor

- Difícil de construir
- Caro
- Aguanta miles de usuarios

Decisión: la A. Es un solo colegio y hay poco tiempo. Si el sistema fuera para todo el país, ganaría la B.

Complejidad arquitectónica y ADL

Cuántas dependencias tiene el diseño, y con qué lenguaje se escribe

Complejidad arquitectónica

Mide cuántas dependencias hay entre las partes de un sistema. A más dependencias, más difícil es cambiar una parte sin romper las otras.

Dependencia de uso

la interfaz no funciona sin el módulo de cálculo

Dependencia de datos

dos módulos leen la misma tabla de notas

Dependencia de orden

hay que registrar la nota antes de calcular el promedio

Regla: menos dependencias entre las partes = sistema más fácil de corregir y de ampliar.

Se mide contando cuántas dependencias tiene cada componente.

Lenguajes de descripción (ADL)

Un ADL es un lenguaje hecho solo para describir arquitecturas. Tiene símbolos y reglas propias, de modo que el diseño quede escrito sin dobles interpretaciones.

¿Para qué sirven?

- Describir componentes y conexiones sin ambigüedades.
- Revisar el diseño antes de escribir una línea de código.
- Comparar dos arquitecturas con el mismo lenguaje.
- Documentar el sistema para quien llegue después.

Ejemplos

UML

ArchiMate

Wright

Acme

En clase el más usado es UML, con sus diagramas de componentes y de clases.

Del flujo de datos a los módulos

Maapeo de transformación: convertir el recorrido de los datos en el diseño de las piezas

Mapear es traducir el camino que siguen los datos a una estructura de módulos (un módulo es una parte del programa con una tarea propia). Cada **burbuja** del flujo de datos es una tarea; el mapeo agrupa esas tareas en módulos y el resultado se dibuja con un **diagrama de componentes UML**: cada módulo es un componente y las flechas muestran quién depende de quién.

